JAVA PROGRAMMING CLASS · PORTFOLIO

PRJ — Institutional Post Assessment

JCasheesh!

SECURE • VIBE CHECKED

PROJECT TITLE

JCasheesh! Bank

A Spring Boot e-wallet web application

Student Name:	Julito III T. Tiro
Batch and Year:	B12 2026
Qualification:	Programming (JAVA) NC III (PRJ)
Scholarship:	Training for Work Scholarship Program (TWSP)
TESDA Office:	TESDA Quezon City
Training Center:	Center for International Industries Competence Corp. (CIICC)
Instructor:	Mr. Marco Yimyaem
Date Submitted:	October 2026

Repository: github.com/jiiitiro/JCasheeshBank

MY REPOSITORY PROFILE

JCasheesh! Bank Project

Repository link of the project:

```
https://github.com/jiiitiro/JCasheeshBank
```

Web-based banking app with user registration, email verification, cash-in, peer-to-peer transfers, receipts and transaction history.

Project Description

JCasheesh! Bank is a full-stack banking project built to practise Java web development with Spring Boot. It works like a basic e-wallet: users register with their mobile number, email and a 4-digit PIN, verify their email, then log in to cash in, send money to another user ("Express Send"), view printable receipts and browse their transaction history.

Compared with a console or desktop bank app, this project runs in the browser, stores its data in PostgreSQL, sends real emails for verification and PIN recovery, and ships as Docker containers so it can run on any machine with one command.

> **Disclaimer:** JCasheesh! Bank is a learning project. It is not connected to any real bank or e-wallet service, and no real money is involved.

Tools Used

- **IntelliJ IDEA** — for writing, running and debugging the Java code
- **Java 17 & Spring Boot 3.5.6** — the application framework (Spring Web, Spring Data JPA, Spring Security, Spring Mail)
- **Thymeleaf** — server-side HTML templates for every page
- **Tailwind CSS** (via CDN) — the glassmorphism user interface
- **PostgreSQL 15** — stores users and transactions
- **Hibernate / JPA** — maps the Java classes to database tables
- **BCrypt** — hashes the PIN before it is saved
- **Gmail SMTP (JavaMailSender)** — sends verification and PIN-reset emails in the background
- **Lombok** — generates getters, setters and constructors
- **Maven 3.9** — build and dependency management
- **Docker & Docker Compose** — runs the app and the database together; **Railway** for cloud deployment
- **Git & GitHub** — version control and source hosting

HOW IT FITS TOGETHER

System Overview

The app is split into layers. Each request from the browser goes to a controller, which calls a service for the business rules, which reads and writes the database through a repository.

Browser
Thymeleaf pages styled with Tailwind CSS. Forms carry a CSRF token.

Controllers
AuthController, DashboardController, CashInController, TransferController

Services
UserService, TransactionService, EmailService, validation services

Repositories
UserRepository and TransactionRepository → PostgreSQL

request → controller → service → repository → database

Main Components

Component	What it does
Login / Register page	The entry point. One page with two tabs: *Sign In* (mobile number + PIN) and *Register* (full name, mobile number, email, PIN).
Email verification	A link is emailed after registering. Unverified accounts cannot log in.
Forgot PIN / Reset PIN	Sends a reset link that expires after 15 minutes, then lets the user set a new 4-digit PIN.
Dashboard	Shows the balance, mobile number, shortcuts to Cash In, Transfer and Transactions, and the 10 most recent transactions.
Cash In	Three steps: enter an amount (₱5.00–₱10,000.00), confirm, then see the receipt.
Express Send (Transfer)	Send money to another user's mobile number. The receiver's name is partly hidden on the confirm screen, and the user must tick a box before sending.
Receipts	Each cash-in and transfer has its own receipt page with a reference number and timestamp.
Transaction History	Lists every cash-in, sent and received transfer of the logged-in user.
Error page	A friendly page for unexpected errors. Internal details are only written to the server log.

Project Structure

```
src/main/java/com/jcashbank/
├── config/        DataInitializer (seed users), SecurityConfig (CSRF)
├── controller/    AuthController, DashboardController, CashInController,
│                  TransferController, SessionUserProvider, GlobalExceptionHandler
├── exception/     BankingException
├── model/         User, Transaction, TransactionType
├── repository/    UserRepository, TransactionRepository
└── service/       UserService, TransactionService, EmailService,
                   UserValidationService, TransactionValidationService
src/main/resources/
├── templates/     Thymeleaf pages (login, dashboard, cash-in, transfer, receipts, PIN reset)
└── static/        CSS and images
```

DATA

Database Schema

LINK:

`https://github.com/jiiitiro/JCasheeshBank/tree/main/src/main/java/com/jcashbank/model`

The app uses a PostgreSQL database with two tables. Hibernate creates and updates them from the `User` and `Transaction` classes. The `user_id` column in `transactions` references `id` in `users` (one-to-many): each user can have many transactions.

Table: users

Column	Type	Notes
`id`	BIGINT, PK	Auto-generated
`mobile_number`	VARCHAR(15)	Unique, required. Starts with 09, 11 digits
`pin`	VARCHAR(100)	BCrypt hash of the 4-digit PIN
`full_name`	VARCHAR(150)	Required
`email`	VARCHAR(150)	Unique, required
`balance`	NUMERIC(15,2)	Starts at 0.00
`failed_login_attempts`	INT	Locks the account at 3
`account_locked, lock_time`	BOOLEAN, TIMESTAMP	Lock state
`email_verified`	BOOLEAN	Must be true to log in
`verification_token, verification_token_expiry`	VARCHAR, TIMESTAMP	Email verification link
`reset_token, reset_token_expiry`	VARCHAR, TIMESTAMP	PIN-reset link (15 minutes)

Table: transactions

Column	Type	Notes
`id`	BIGINT, PK	Auto-generated
`user_id`	BIGINT, FK → users.id	Owner of the record
`type`	VARCHAR(30)	`CASH_IN`, `TRANSFER_SENT` or `TRANSFER_RECEIVED`
`amount`	NUMERIC(15,2)	Required
`details`	VARCHAR(255)	e.g. “Transfer sent to 09171234567”
`transaction_date`	TIMESTAMP	Set automatically when saved
`reference_number`	VARCHAR, unique	Format `0000 000 000000`; the receiver's copy ends in `-R`

A transfer writes two rows: a `TRANSFER_SENT` row for the sender and a `TRANSFER_RECEIVED` row for the receiver.

THE PROJECT SOURCE CODE

Source Code

SOURCE CODE (AuthController)

https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/controller/AuthController.java

Handles login, logout, registration, email verification and the forgot/reset PIN pages. After a successful login it gives the user a new session ID so an old one can't be reused.

```
@PostMapping("/login")
public String login(@RequestParam String mobileNumber, @RequestParam String pin,
                    HttpServletRequest request, HttpSession session, Model model) {
    try {
        User user = userService.authenticate(mobileNumber.trim(), pin.trim());
        // New session ID on login so a session ID planted before login is useless
        request.changeSessionId();
        session.setAttribute(SESSION_USER_ID, user.getId());
        return "redirect:/dashboard";
    } catch (BankingException ex) {
        model.addAttribute("error", ex.getMessage());
        return "login";
    }
}
```

SOURCE CODE (UserService)

https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/service/UserService.java

Registers users, checks the PIN with BCrypt, counts failed attempts and locks the account on the third failure, then emails a reset link.

```
if (!passwordEncoder.matches(pin.trim(), user.getPin())) {
    int attempts = user.getFailedLoginAttempts() + 1;
    user.setFailedLoginAttempts(attempts);

    // On 3rd failure for an existing account, lock and send reset email
    if (attempts >= 3) {
        user.setAccountLocked(true);
        user.setLockTime(LocalDateTime.now());
        String token = UUID.randomUUID().toString();
        user.setResetToken(token);
        user.setResetTokenExpiry(LocalDateTime.now().plusMinutes(15));
        userRepository.saveAndFlush(user);
        emailService.sendEmailAsync(user.getEmail(), subject, body);
        throw new BankingException("Maximum 3 failed login attempts reached. Account locked...");
    }
    userRepository.saveAndFlush(user);
    throw new BankingException("Mobile number or PIN is incorrect. Attempts remaining: " + (3 - attempts));
}
```

THE PROJECT SOURCE CODE

SOURCE CODE (TransactionService)

https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/service/TransactionService.java

Moves the money. Both accounts are locked in the database (`SELECT … FOR UPDATE`) during a transfer, always lowest ID first, so two transfers at the same moment can't overdraw an account or block each other.

```
@Transactional
public Transaction transfer(Long senderId, String receiverMobileNumber, BigDecimal amount) {
    transactionValidationService.validatePositiveAmount(amount);
    Long receiverId = userRepository.findByMobileNumber(receiverMobileNumber)
            .map(User::getId)
            .orElseThrow(() -> new BankingException("Receiver account was not found."));
    if (senderId.equals(receiverId)) {
        throw new BankingException("You cannot transfer money to your own account.");
    }
    // Lock both rows in ID order so two users sending to each other at once can't deadlock
    User sender, receiver;
    if (senderId < receiverId) { sender = lockUser(senderId, ...); receiver = lockUser(receiverId, ...); }
    else                       { receiver = lockUser(receiverId, ...); sender = lockUser(senderId, ...); }

    // Balance is re-read under the lock, so concurrent transfers can't both pass this check
    if (sender.getBalance().compareTo(amount) < 0) {
        throw new BankingException("Insufficient balance.");
    }
    sender.setBalance(sender.getBalance().subtract(amount));
    receiver.setBalance(receiver.getBalance().add(amount));
    // ... create TRANSFER_SENT and TRANSFER_RECEIVED records with reference numbers
    transactionRepository.save(receivedTx);
    return transactionRepository.save(sentTx);
}

private User lockUser(Long userId, String notFoundMessage) {
    User user = userRepository.findById(userId)
            .orElseThrow(() -> new BankingException(notFoundMessage));
    entityManager.refresh(user, LockModeType.PESSIMISTIC_WRITE);
    return user;
}
```

SOURCE CODE (TransactionValidationService)

https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/service/TransactionValidationService.java

Amount rules are checked on the server, not only in the browser, so they can't be skipped by sending a request directly.

```
private static final BigDecimal CASH_IN_MIN = new BigDecimal("5.00");
private static final BigDecimal CASH_IN_MAX = new BigDecimal("10000.00");

public void validateCashInAmount(BigDecimal amount) {
    if (amount == null || amount.compareTo(CASH_IN_MIN) < 0 || amount.compareTo(CASH_IN_MAX) > 0) {
        throw new BankingException("Cash in amount must be between ₱5.00 and ₱10,000.00.");
    }
}
```

THE PROJECT SOURCE CODE

SOURCE CODE (SecurityConfig)

`https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/config/SecurityConfig.java`

Turns on Spring Security's CSRF protection. Every Thymeleaf form gets a hidden token, and requests without it are rejected with 403. Login and sessions are still handled by the app's own controllers.

```
http
    .authorizeHttpRequests(auth -> auth.anyRequest().permitAll())
    // CSRF stays enabled (default); th:action forms get the hidden _csrf field automatically
    .formLogin(AbstractHttpConfigurer::disable)
    .httpBasic(AbstractHttpConfigurer::disable)
    .logout(AbstractHttpConfigurer::disable);
```

SOURCE CODE (Models)

`https://github.com/jiiitiro/JCasheeshBank/blob/main/src/main/java/com/jcashbank/model/Transaction.java`

Each transaction gets a reference number and a timestamp automatically just before it is saved.

```
@PrePersist
public void prePersist() {
    if (this.referenceNumber == null || this.referenceNumber.isEmpty()) {
        this.referenceNumber = generateFormattedRefNo();   // e.g. "4821 305 774120"
    }
    if (this.transactionDate == null) {
        this.transactionDate = LocalDateTime.now();
    }
}
```

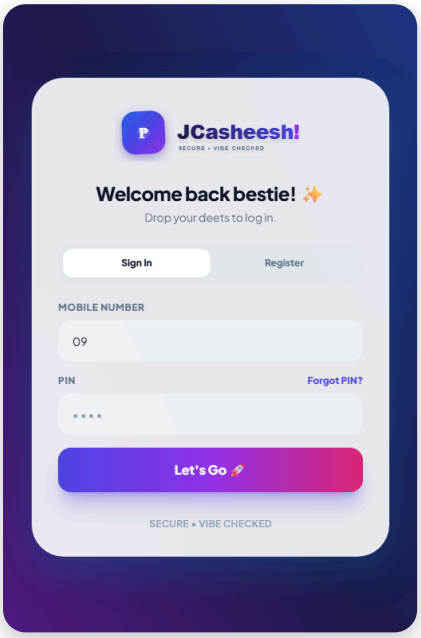
Other source files

File	Role
`CashInController`	Cash-in form, confirm step, execute, and receipt page
`TransferController`	Transfer form, confirm step (masked name), execute, and receipt page
`DashboardController`	Dashboard (latest 10) and full transaction history
`EmailService`	Sends verification and PIN-reset emails with `@Async`
`GlobalExceptionHandler`	Shows friendly messages; logs unexpected errors on the server

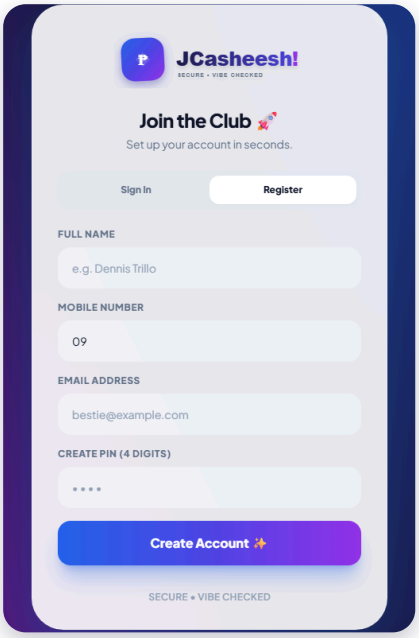
OUTPUT OF THE SOURCE CODE

Logging In & Registering

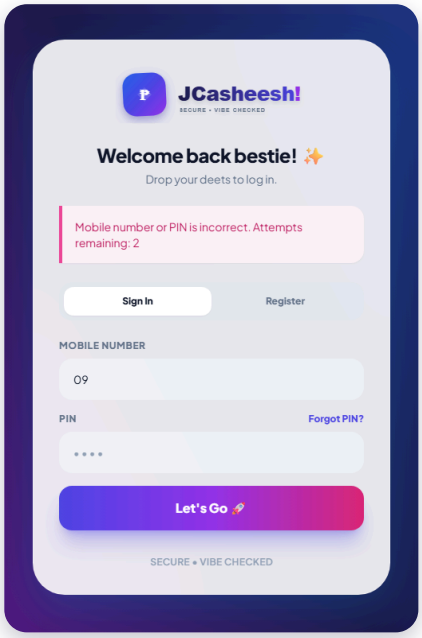
The login page is the entry point. Users switch between *Sign In* and *Register* with the tabs. Mobile numbers always start with **09** and are limited to 11 digits; PINs accept only 4 numbers. After a successful login the dashboard opens. If the details are wrong, a message shows how many attempts are left.



1 Sign In
Mobile number and 4-digit PIN.

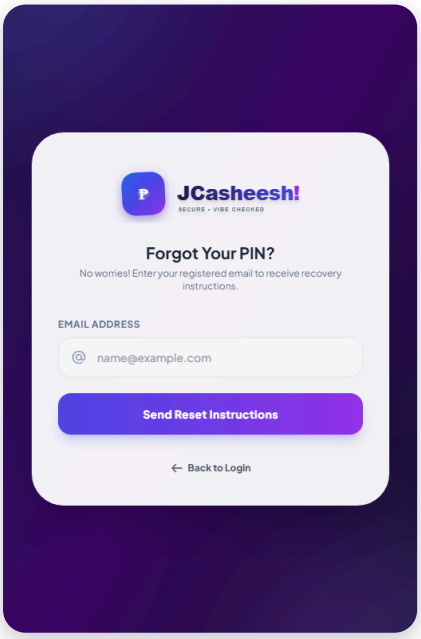


2 Register
Name, mobile number, email and PIN. A verification email is sent.

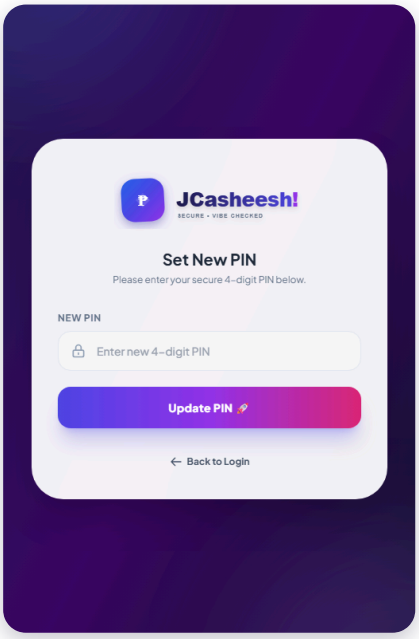


3 Wrong PIN
The 3rd failed attempt locks the account and emails a reset link.

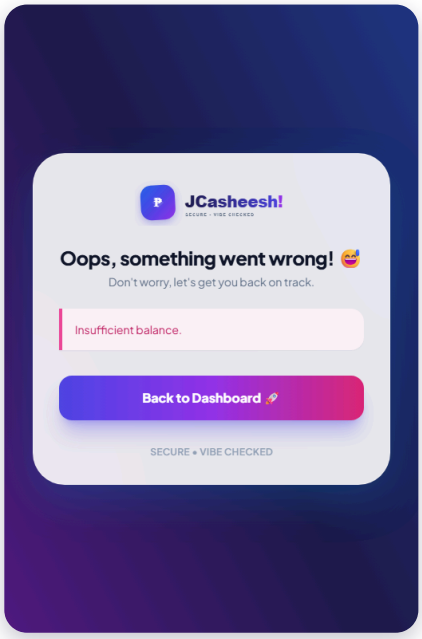
Forgot & Reset PIN



4 Forgot PIN
Enter the registered email to get a reset link.



5 Set New PIN
The link expires after 15 minutes. The new PIN must differ from the old one.

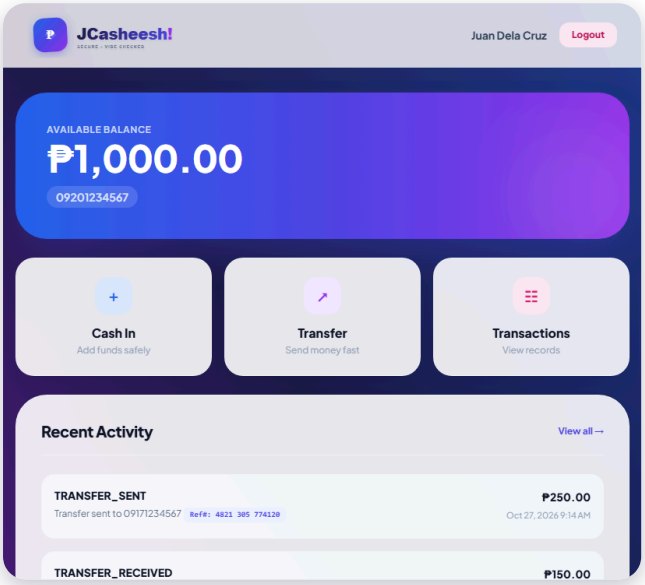


6 Error page
Shown for unexpected problems, without internal details.

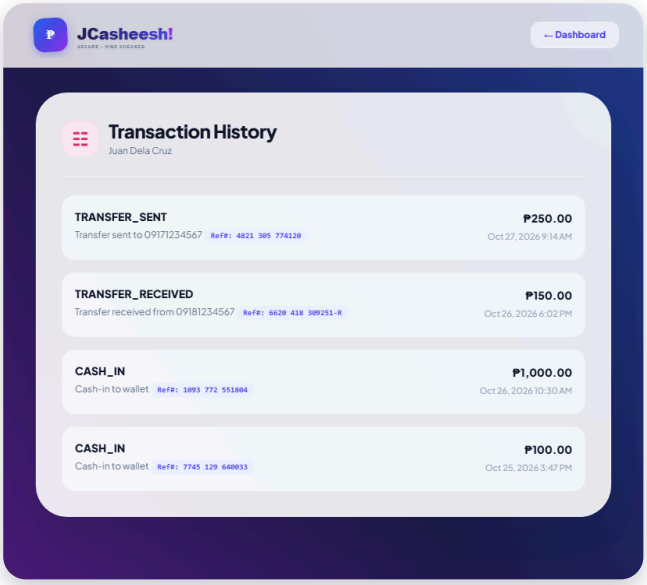
OUTPUT OF THE SOURCE CODE

Dashboard & Transaction History

After logging in, the dashboard shows the available balance and the user's mobile number, three shortcuts (Cash In, Transfer, Transactions) and the 10 most recent transactions. *View all* opens the full history.



7 Dashboard
Balance, shortcuts and recent activity.



8 Transaction History
Every cash-in, sent and received transfer, each with a reference number.

Choosing Options

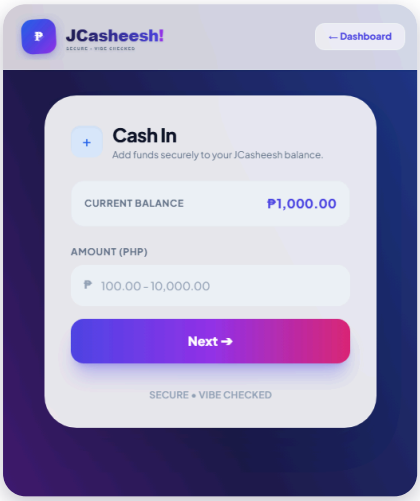
Messages appear when the user enters something the app can't accept, for example when:

- the receiver's mobile number doesn't belong to any account (“Receiver account was not found.”)
- the user tries to send money to their own number (“You cannot transfer money to your own account.”)
- the amount is zero or negative (“Amount must be greater than zero.”)
- the balance is too low for a transfer (“Insufficient balance.”)
- a cash-in is below ₱5.00 or above ₱10,000.00
- the account hasn't been verified by email yet, or is locked after 3 failed logins

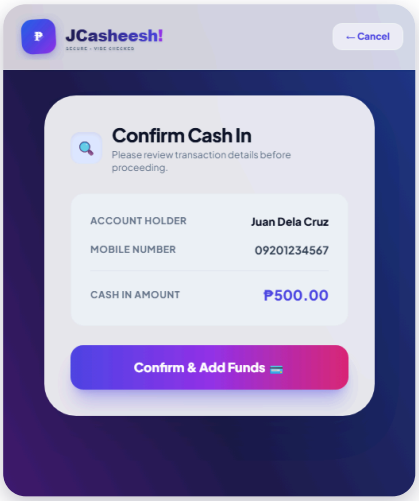
OUTPUT OF THE SOURCE CODE

Cash In

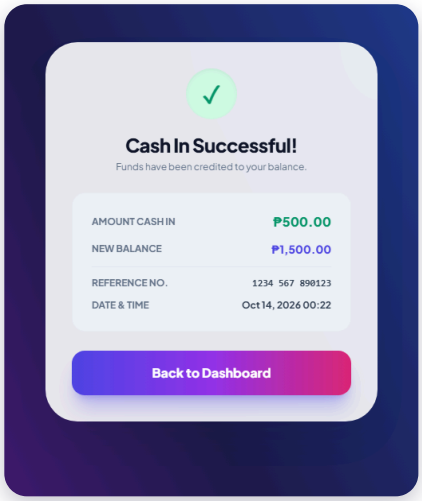
Cash-in takes three steps. The user enters an amount from ₱5.00 to ₱10,000.00, checks the details on the confirm screen, then sees a receipt with the new balance and a reference number. The receipt has its own address, so refreshing it never adds the money twice.



9 Enter amount
Shows the current balance.



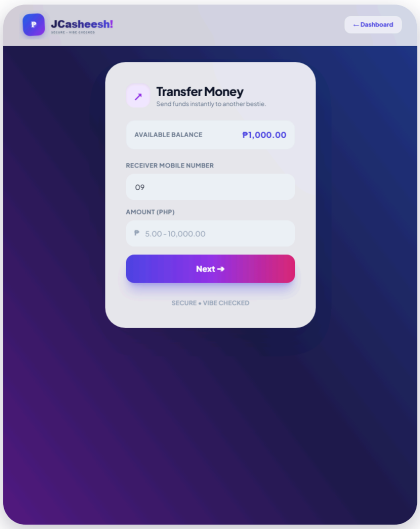
10 Confirm
Account holder, mobile number and amount.



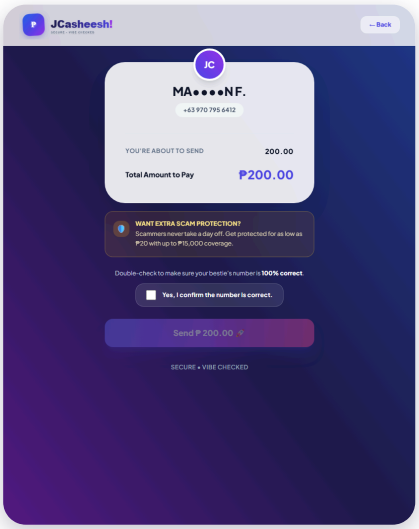
11 Receipt
Amount, new balance, reference number and date.

Express Send (Transfer)

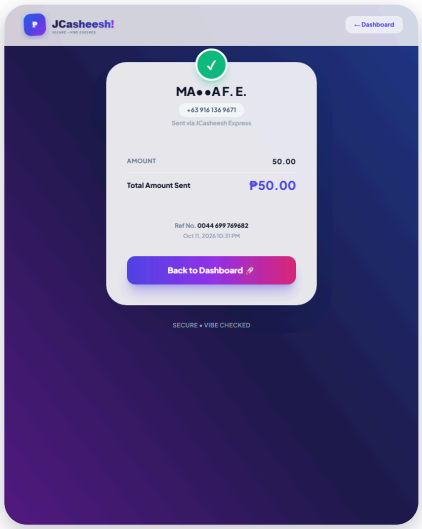
The user enters the receiver's mobile number and the amount. The confirm screen shows the receiver's name partly hidden for privacy, and the *Send* button only works after ticking “Yes, I confirm the number is correct.” The receipt can be printed, and only the owner of the transaction can open it.



12 Transfer form
Receiver number and amount.



13 Confirm
Masked name and confirmation checkbox.



14 Receipt
Amount sent, reference number and date.

Screens on pages 8–10 were rendered from the project's Thymeleaf templates with sample data.

ACCOUNT PROTECTION

Security Features

Feature	How it works
Hashed PINs	PINs are saved as BCrypt hashes, never as plain text.
Email verification	New accounts must click the emailed link before they can log in.
Brute-force lockout	Failed logins are counted per account. The 3rd failure locks it and emails a PIN-reset link.
Expiring reset links	Reset tokens are random and expire after 15 minutes. The new PIN is checked on the server.
CSRF protection	Every form carries a token. Requests from other websites are rejected with 403.
Session fixation protection	The session ID changes on every successful login.
Safe concurrent transfers	Account rows are locked during cash-in and transfers, so two transfers at once can't overdraw an account.
Server-side limits	Cash-in limits and positive-amount checks are enforced in the service layer.
Refresh-safe receipts	After paying, the app redirects to a receipt page, so a refresh doesn't repeat the payment.
No error leaks	Unexpected errors are logged on the server; users see a generic message.

The full list of fixes and remaining open items is in `SECURITY.md` in the repository.

Running the Project

With Docker Compose

Create a `.env` file in the project root with `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `MAIL_USERNAME`, `MAIL_PASSWORD` and `APP_BASE_URL`, then run:

```
git clone https://github.com/jiiitiro/JCasheeshBank.git
cd JCasheeshBank
docker compose up --build
```

Open **http://localhost:8081** in the browser.

From IntelliJ

Start only the database with `docker compose up -d postgres` (available on `localhost:5434`), then run `JcashBankApplication`. The app listens on port 8081.

Test accounts

On first start with an empty database, `DataInitializer` creates three accounts: `09171234567` / PIN `1234`, `09181234567` / PIN `5678`, and `09201234567` / PIN `9999` (starting balance ₱1,000.00).



Thank you very much!

https://github.com/jiiitiro/JCasheeshBank